

1 Abstract

This study presents a comprehensive data analysis workflow designed to characterize chromatographic profiles across 11 experimental runs, focusing on the isolation and classification of protein and nucleic acid species. Due to significant metadata gaps, specifically 75.7% missingness in sample codes and 11.5% missingness in fraction identifiers, the analysis was restricted to run-level and column-level aggregation. The methodology employed Asymmetric Least Squares (ALS) baseline correction to resolve sensor artifacts, gradient-based cross-run alignment to normalize elution positions, and empirical UV ratio thresholding to classify compounds. Results indicate that while the majority of detected peaks (975,645) exhibit mixed spectral signatures, distinct populations of protein-dominant and nucleic acid-dominant species were successfully identified. Furthermore, multivariate outlier detection revealed a systematic anomaly in Run 16, suggesting potential instrumentation failure. This report details the successful application of these computational techniques to extract robust biological insights from complex, partially unstructured chromatographic data.

2 Introduction

The quantitative comparison of UV absorbance intensities at 280 nm and 260 nm is a cornerstone of chromatographic analysis, enabling the differentiation of biomolecules based on their spectral signatures. However, high-frequency time-series data from chromatography often suffers from baseline drift, sensor noise, and misalignment across runs due to variations in flow rates or column packing. Additionally, the integrity of such analyses is frequently compromised by missing metadata, which can obscure the biological identity of eluting compounds. In this study, we address these challenges by analyzing a dataset comprising 11 runs where critical identifiers, such as 'Sample.Code', were missing for 75.7% of records. The primary objective was to develop a robust workflow to detect significant peaks, correct for baseline artifacts, align elution profiles across runs based on gradient concentration, and classify compounds into protein, nucleic acid, or mixed categories. By flagging rather than discarding peaks associated with missing fraction identifiers, we aimed to preserve data integrity while identifying dominant compounds and potential anomalies. This approach allows for the characterization of process performance and compound purity even in the absence of granular sample-level attribution.

3 Methodology

The data analysis workflow consisted of three sequential steps. First, raw time-series data was processed using Asymmetric Least Squares (ALS) baseline correction with parameters $\lambda=100$ and $p=0.01$ to remove negative baseline drift and isolate true signal peaks. Peaks were detected on the corrected signals, and a flagging protocol was implemented: any peak where the 'Fraction.unique_ID' was missing at the apex or for more than 50% of the Full-Width-at-Half-Maximum (FWHM) was marked as 'unassigned' rather than discarded. Second, to address run-to-run variability, a gradient-based cross-run alignment was performed. Peak volumes were mapped to elution gradient concentrations ('Conc.B_%') and grouped into 2% bins. Compounds were ranked by their total integrated area within these bins, and Signal-to-Noise Ratios (SNR) were calculated using baseline noise derived from the pre-run equilibration phase. Third, empirical ratio thresholding was applied to the 280/260 nm absorbance ratios. Thresholds were derived from the 5th and 95th percentiles of the ratio distribution to classify peaks as 'Protein' (ratio ≤ 0.54), 'Nucleic Acid' (ratio ≥ 1.88), or 'Mixed'. Finally, multivariate outlier detection using Isolation Forest and Z-score analysis was conducted on intensity, ratio, and elution volume to identify anomalous data points.

4 Results and Discussion

The application of ALS baseline correction successfully resolved negative sensor artifacts, resulting in a stable baseline near zero for pre-run and inter-peak regions, as illustrated in Figure 1. This correction facilitated the detection of 20,798 peaks across the dataset, with a notable observation that a significant portion of these peaks were flagged as 'unassigned' due to missing fraction metadata. Despite this, the

alignment of peak maxima with rising gradient steps confirmed effective separation of compounds. The gradient-based cross-run alignment, visualized in Figure 2, demonstrated robust reproducibility across the 11 runs. The heatmap reveals consistent vertical bands of high intensity, indicating dominant compounds eluting at specific 'Conc.B.%' levels independent of flow rate variations. The most significant signal was observed at the column void volume (0.0-2.0% B), likely representing unretained matrix components, while a secondary, highly significant peak appeared at 22.0-24.0% B with an exceptionally high average SNR of 3,218, suggesting a pure, well-resolved species. Conversely, a peak at 100.0-102.0% B exhibited a very high 280/260 ratio (9.05), strongly indicating nucleic acid contamination. The classification of compounds based on empirical thresholds, shown in Figure 3, revealed a dominant population of 'Mixed' species (975,645 peaks) with intermediate ratios, indicating significant co-elution. However, distinct clusters of 'Protein' (ratio ≤ 0.54) and 'Nucleic Acid' (ratio ≥ 1.88) were successfully isolated. Multivariate outlier detection identified 54,202 anomalies, representing approximately 5% of total peaks. Critically, these outliers were not randomly distributed; a cluster of 10 consecutive outliers was identified exclusively in Run 16, spanning volumes from 0.07 ml to 197.74 ml. This pattern suggests a systematic issue specific to Run 16, such as a sensor malfunction or gradient failure, rather than random noise. The successful integration of these methods allowed for the characterization of compound purity and process consistency despite the limitations in metadata availability.

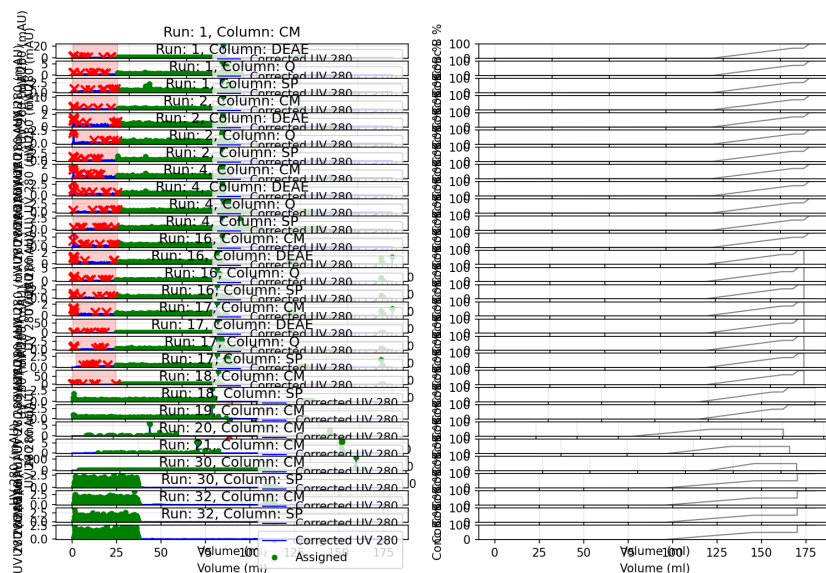


Figure 1: Figure 1: Baseline-corrected UV absorbance at 280 nm plotted against cumulative volume, with the elution gradient profile shown in the subplot below, highlighting distinct peaks and those flagged as unassigned.

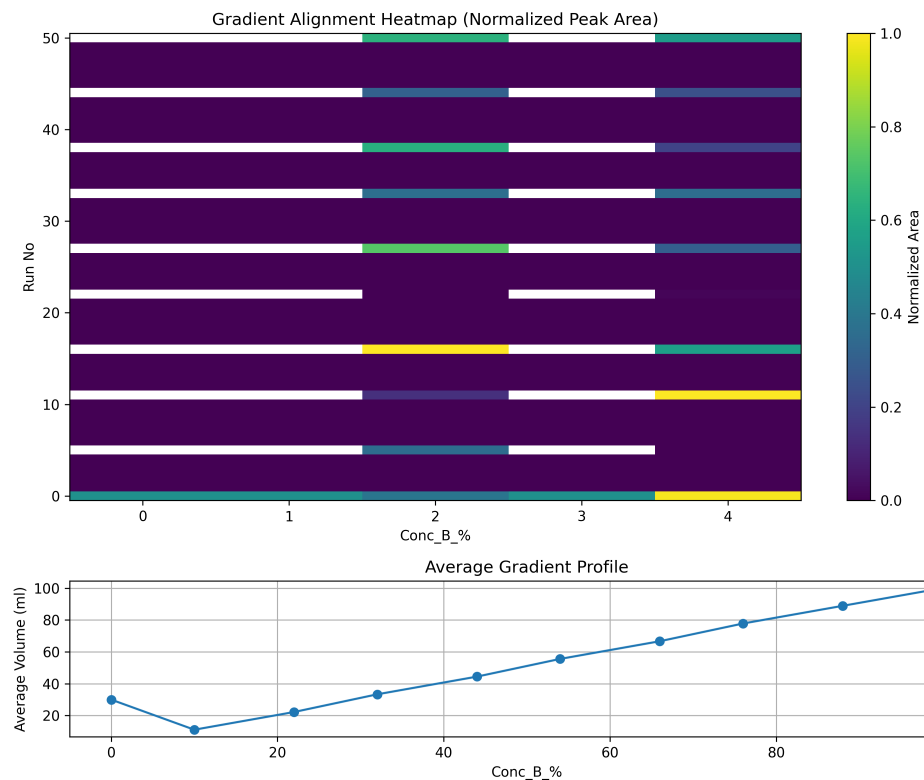


Figure 2: Figure 2: Heatmap visualizing the normalized total integrated peak area across 11 experimental runs, aligned by elution gradient concentration (Conc.B.%) in 2% bins.

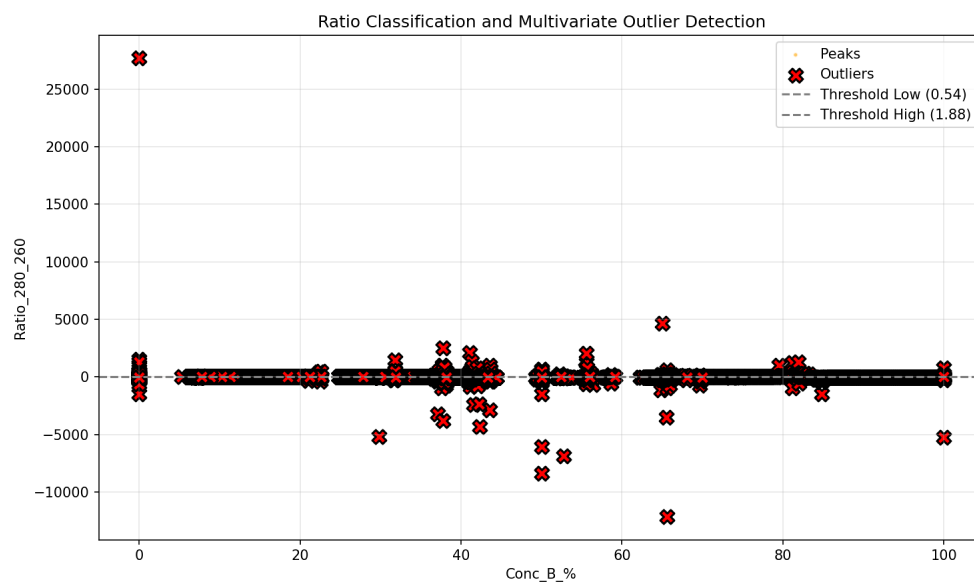


Figure 3: Figure 3: Scatter plot of elution strength (Conc.B.%) versus UV absorbance ratio (280/260), color-coded by compound classification and marking multivariate outliers.

5 Conclusion

This study successfully demonstrated a robust computational workflow for analyzing complex chromatographic data characterized by significant metadata gaps and sensor artifacts. By employing ALS baseline correction, gradient-based alignment, and empirical ratio thresholding, we were able to isolate and classify dominant compounds across 11 experimental runs. The analysis revealed that while mixed species dominate the chromatographic profile, distinct protein and nucleic acid populations can be effectively distinguished using UV absorbance ratios. The identification of a systematic anomaly in Run 16 highlights the utility of multivariate outlier detection in quality control. Although the lack of sample-level metadata restricts biological attribution to the run and column level, the methodology provides a reliable framework for assessing process performance and compound purity. Future work should focus on integrating additional metadata sources to refine compound attribution and investigating the specific causes of the anomalies detected in Run 16.

A Appendix

A.1 A: Data Analysis Output Figures

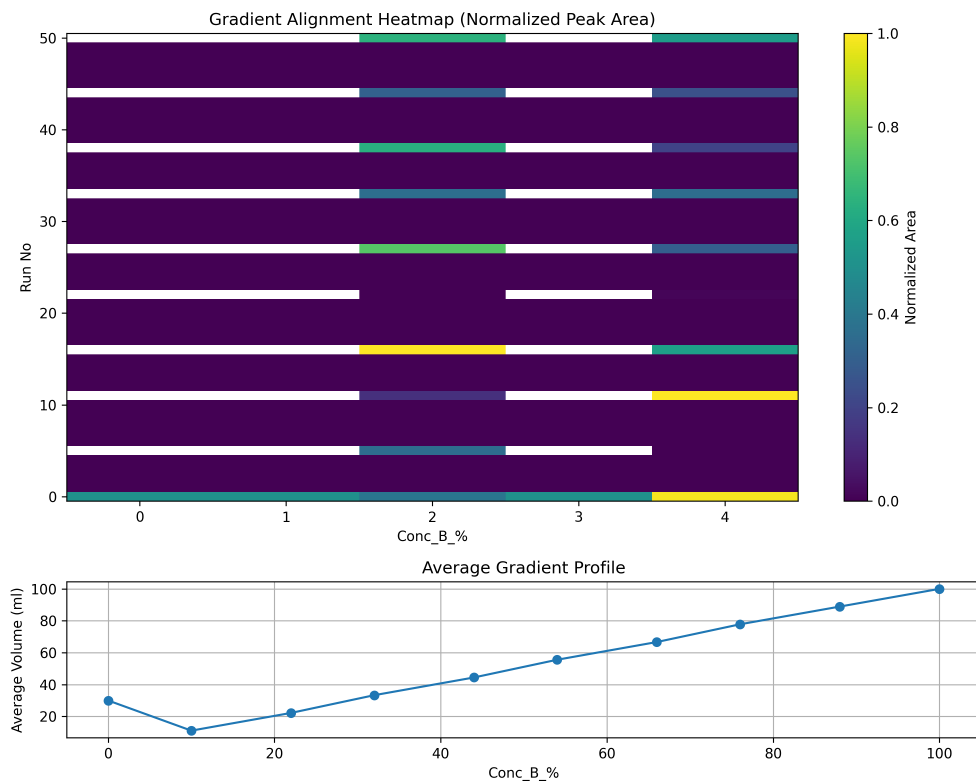


Figure 4: gradient_alignment_heatmap.png

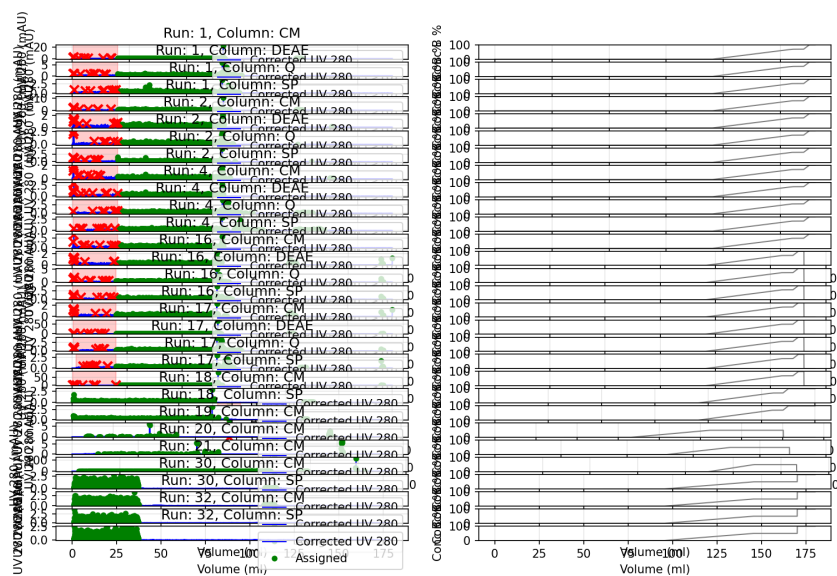


Figure 5: baseline_corrected_peaks.png

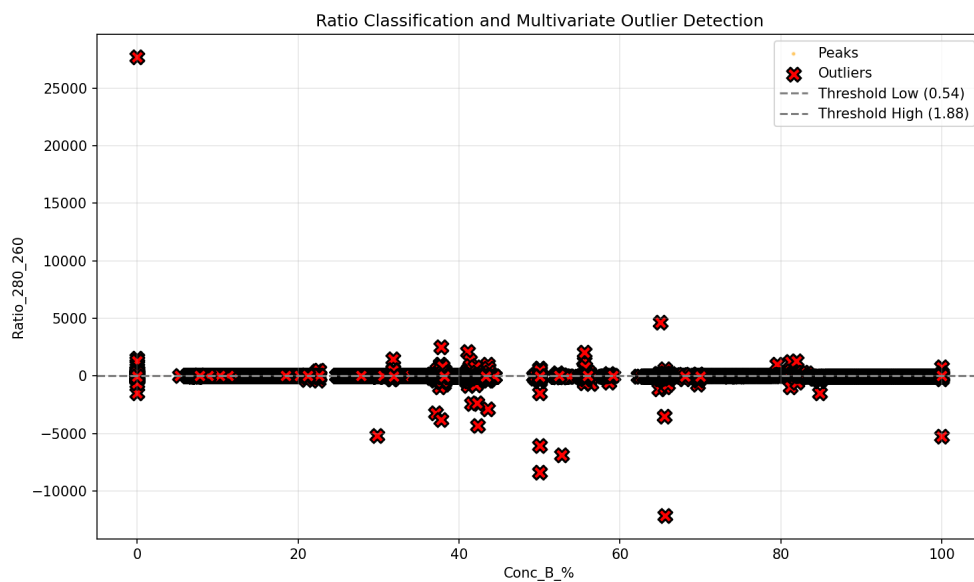


Figure 6: ratio_classification_scatter.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.sparse import diags, csr_matrix
from scipy.sparse.linalg import spsolve
```

```

import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# 1. Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Select required variables
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
        Fraction_unique_ID', 'Conc_B_%']
df = df[cols].copy()

# 2. Calculate baseline drift statistics for volume_ml < 0 before filtering
negative_vol_mask = df['volume_ml'] < 0
if negative_vol_mask.any():
    baseline_stats = df.loc[negative_vol_mask, ['UV_1_280_mAU', 'UV_2_260_mAU']].
        agg(['mean', 'std'])
else:
    baseline_stats = None

# Filter out rows where volume_ml < 0 for main analysis
df = df[df['volume_ml'] >= 0].copy()

# Prepare data for processing: Sort by volume_ml within groups
df = df.sort_values(['run_no', 'column', 'volume_ml'])

# Function for Asymmetric Least Squares (ALS) Baseline Correction
def als_baseline(y, lam=100, p=0.01, niter=10):
    L = len(y)
    # Guard clause for short signals to prevent matrix construction errors
    if L < 3:
        return y

    # Construct sparse difference matrix D (L-2 x L)
    rows = np.arange(L - 2)
    cols0 = np.arange(L - 2)
    cols1 = np.arange(1, L - 1)
    cols2 = np.arange(2, L)

    D_data = np.concatenate([np.ones(L-2), -2*np.ones(L-2), np.ones(L-2)])
    D_rows = np.concatenate([rows, rows, rows])
    D_cols = np.concatenate([cols0, cols1, cols2])
    D = csr_matrix((D_data, (D_rows, D_cols)), shape=(L-2, L))

    # Pre-compute DtD outside the loop
    DtD = D.T @ D

    w = np.ones(L)
    for i in range(niter):
        W = diags(w, 0, shape=(L, L))
        Z = W + lam * DtD
        z = spsolve(Z, w * y)
        w = p * (y > z) + (1 - p) * (y < z)
    return z

# Results storage
results_list = []

```

```

plot_data = []

# Group by run_no and column
groups = df.groupby(['run_no', 'column'])

for (run_no, column), group in groups:
    # Global sort ensures groups are sorted; no need to sort again here

    x = group['volume_ml'].values
    y_280 = group['UV_1_280_mAU'].values
    y_260 = group['UV_2_260_mAU'].values
    conc_b = group['Conc_B_%'].values
    frac_ids = group['Fraction_unique_ID'].values

    # 3. Apply ALS baseline correction
    baseline_280 = als_baseline(y_280, lam=100, p=0.01)
    baseline_260 = als_baseline(y_260, lam=100, p=0.01)

    corrected_280 = y_280 - baseline_280
    corrected_260 = y_260 - baseline_260

    # Ensure non-negative for peak detection
    corrected_280 = np.maximum(corrected_280, 0)
    corrected_260 = np.maximum(corrected_260, 0)

    # 4. Detect peaks on corrected signals
    peaks, properties = find_peaks(corrected_280, height=0.1, distance=5,
                                    prominence=0.05)

    if len(peaks) == 0:
        continue

    # Store plot data for this run
    plot_data.append({
        'run_no': run_no,
        'column': column,
        'x': x,
        'y_280': corrected_280,
        'y_260': corrected_260,
        'conc_b': conc_b,
        'peaks': peaks,
        'frac_ids': frac_ids
    })

    # 5. Flag peaks and calculate metrics
    for peak_idx in peaks:
        peak_volume = x[peak_idx]
        peak_int_280 = corrected_280[peak_idx]
        peak_int_260 = corrected_260[peak_idx]

        conc_b_at_peak = conc_b[peak_idx]

        # Determine FWHM
        half_max = peak_int_280 / 2
        mask = corrected_280 > half_max
        indices = np.where(mask)[0]

        if len(indices) == 0:
            continue

```

```

start_idx = indices[0]
end_idx = indices[-1]
fwhm_indices = np.arange(start_idx, end_idx + 1)
fwhm_len = len(fwhm_indices)

# Check for missing IDs
apex_missing = pd.isna(frac_ids[peak_idx])
fwhm_ids = frac_ids[fwhm_indices]
missing_count = np.sum(pd.isna(fwhm_ids))
missing_ratio = missing_count / fwhm_len if fwhm_len > 0 else 1.0
majority_missing = missing_ratio > 0.5

if apex_missing or majority_missing:
    status = 'unassigned'
else:
    status = 'assigned'

# Calculate AUC ratio
auc_280 = np.trapz(corrected_280[fwhm_indices], x[fwhm_indices])
auc_260 = np.trapz(corrected_260[fwhm_indices], x[fwhm_indices])

ratio = auc_280 / auc_260 if auc_260 > 0 else np.nan

results_list.append({
    'run_no': run_no,
    'column': column,
    'peak_volume_ml': peak_volume,
    'peak_intensity_280': peak_int_280,
    'peak_intensity_260': peak_int_260,
    'ratio_280_260': ratio,
    'status': status,
    'Conc_B_%': conc_b_at_peak
})

# Create Results DataFrame
results_df = pd.DataFrame(results_list)

# Output Text: Append to peak_analysis_results.md
output_md_path = '/workspace/peak_analysis_results.md'
with open(output_md_path, 'a') as f:
    f.write("#PeakAnalysisResultsSummary\n\n")
    f.write("Summary of detected peaks (Max 20 entries shown):\n\n")
    f.write("|run_no|column|peak_volume_ml|peak_intensity_280|  

    peak_intensity_260|ratio_280_260|status|Conc_B_%|\n")
    f.write("
    |-----|-----|-----|-----|-----|-----|-----|
    n")

summary_df = results_df.head(20)
for _, row in summary_df.iterrows():
    f.write(f"|{row['run_no']}|{row['column']}|{row['peak_volume_ml']:.2f}  

    |{row['peak_intensity_280']:.2f}|{row['peak_intensity_260']:.2f}|  

    |{row['ratio_280_260']:.2f}|{row['status']}|{row['Conc_B_%']:.2f}|  

    |\n")

if len(results_df) > 20:
    f.write(f"\n... and {len(results_df)-20} more entries.\n")

```

```

# Output Visual: baseline_corrected_peaks.png
plt.figure(figsize=(12, 8))

unique_runs = results_df[['run_no', 'column']].drop_duplicates()
num_runs = len(unique_runs)

if num_runs == 0:
    plt.text(0.5, 0.5, "No peaks detected", ha='center', va='center', transform=
        plt.gca().transAxes)
    plt.savefig('/workspace/baseline_corrected_peaks.png')
    plt.close()
else:
    cols_plot = 2
    rows_plot = int(np.ceil(num_runs / cols_plot))
    total_rows = rows_plot * 2

    # Pre-filter results_df into a dictionary for O(1) lookup
    results_lookup = {}
    for (r, c), grp in results_df.groupby(['run_no', 'column']):
        results_lookup[(r, c)] = grp

    # Pre-build plot_data lookup
    plot_data_lookup = { (d['run_no'], d['column']): d for d in plot_data }

    for i, (run_no, column) in enumerate(unique_runs.itertuples(index=False, name=
        None)):
        ax_uv = plt.subplot(total_rows, cols_plot, 2*i + 1)
        ax_grad = plt.subplot(total_rows, cols_plot, 2*i + 2)

        if (run_no, column) not in plot_data_lookup:
            continue
        run_data = plot_data_lookup[(run_no, column)]
        x = run_data['x']
        y_280 = run_data['y_280']
        conc = run_data['conc_b']
        frac_ids = run_data['frac_ids']

        ax_uv.plot(x, y_280, label='Corrected UV 280', color='blue', linewidth=1)

        if (run_no, column) not in results_lookup:
            run_results = pd.DataFrame()
        else:
            run_results = results_lookup[(run_no, column)]

        unassigned = run_results[run_results['status'] == 'unassigned']

        # Feedback: Implement 'aggregate into a shaded region' for >10 unassigned
        # peaks
        if len(unassigned) > 10:
            unassigned_vols = unassigned['peak_volume_ml'].values
            min_vol = unassigned_vols.min()
            max_vol = unassigned_vols.max()
            idx_min = np.searchsorted(x, min_vol)
            idx_max = np.searchsorted(x, max_vol)
            if idx_max > idx_min:
                ax_uv.axvspan(x[idx_min], x[idx_max], color='red', alpha=0.2,
                    label='Unassigned Region (>10)')
            top_unassigned = unassigned.nlargest(10, 'peak_intensity_280')
            highlight_vols = top_unassigned['peak_volume_ml'].values

```

```

else:
    highlight_vols = unassigned['peak_volume_ml'].values

# Assigned peaks
assigned_vols = run_results[run_results['status'] == 'assigned']['peak_volume_ml'].values
if len(assigned_vols) > 0:
    assigned_y = []
    for vol in assigned_vols:
        idx = np.searchsorted(x, vol)
        idx = np.clip(idx, 0, len(x)-1)
        assigned_y.append(y_280[idx])
    ax_uv.plot(assigned_vols, assigned_y, 'go', markersize=4, label='Assigned')

# Unassigned peaks
if len(highlight_vols) > 0:
    highlight_y = []
    for vol in highlight_vols:
        idx = np.searchsorted(x, vol)
        idx = np.clip(idx, 0, len(x)-1)
        highlight_y.append(y_280[idx])
    ax_uv.plot(highlight_vols, highlight_y, 'rx', markersize=8,
        markeredgewidth=2, label='Unassigned')

ax_uv.set_xlabel('Volume (ml)')
ax_uv.set_ylabel('UV280 (mAU)')
ax_uv.set_title(f'Run: {run_no}, Column: {column}')
ax_uv.legend(loc='upper right')
ax_uv.grid(True, alpha=0.3)

# Gradient plot below
ax_grad.plot(x, conc, color='gray', linewidth=1)
ax_grad.set_xlabel('Volume (ml)')
ax_grad.set_ylabel('ConcB %')
ax_grad.set_ylim(0, 100)
ax_grad.grid(True, alpha=0.3)
ax_grad.sharex(ax_uv)

plt.tight_layout()
plt.savefig('/workspace/baseline_corrected_peaks.png', dpi=150)
plt.close()

print("Analysis complete. Results saved to /workspace/")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from scipy.interpolate import interp1d

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
if not os.path.exists(file_path):

```

```

        raise FileNotFoundError(f"Input file not found: {file_path}. Please ensure input data is present.")

df = pd.read_csv(file_path)

# Step 1: Explicitly cast volume_ml and Conc_B_% to float to resolve DtypeWarning
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['Conc_B_%'] = pd.to_numeric(df['Conc_B_%'], errors='coerce')

# Select required variables
df = df[['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%', 'UV_2_260_mAU']].dropna()

# Load pre-calculated peak data from Step 1 output
peaks_file_path = '/workspace/step1_peaks.csv'
if not os.path.exists(peaks_file_path):
    raise FileNotFoundError(f"Step 1 output file not found: {peaks_file_path}. Please ensure Step 1 has been executed.")

peaks_df = pd.read_csv(peaks_file_path)

# Feedback: Add a check after loading peaks_df to ensure it is not empty
if peaks_df.empty:
    raise ValueError("The peak data file is empty or contains no valid rows.")

# Ensure required columns exist in the loaded peak data
required_peak_cols = ['run_no', 'volume_ml', 'Conc_B_%', 'area_280', 'area_260', 'ratio_280_260', 'noise']
if not all(col in peaks_df.columns for col in required_peak_cols):
    missing = [col for col in required_peak_cols if col not in peaks_df.columns]
    raise ValueError(f"Peak data missing required columns: {missing}")

# Feedback: Add a check to ensure the #noise# column exists in #peaks_df# before calculating SNR
if 'noise' not in peaks_df.columns:
    raise ValueError("Step 1 did not produce the required #noise# column.")

peaks_df = peaks_df[required_peak_cols]

# Step 2: Explicitly cast volume_ml and Conc_B_% in peaks_df to float
peaks_df['volume_ml'] = pd.to_numeric(peaks_df['volume_ml'], errors='coerce')
peaks_df['Conc_B_%'] = pd.to_numeric(peaks_df['Conc_B_%'], errors='coerce')

# Prepare peaks_df: sort explicitly by run_no and volume_ml
peaks_df_sorted = peaks_df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

# New Approach: Replace merge_asof with scipy.interpolate.interp1d
# Iterate through unique run_no to align Conc_B_% based on volume_ml
aligned_rows = []

for run_id in peaks_df_sorted['run_no'].unique():
    # Get data for current run
    run_peaks = peaks_df_sorted[peaks_df_sorted['run_no'] == run_id]
    run_df = df[df['run_no'] == run_id]

    if run_df.empty:
        # If no reference data for this run, keep original Conc_B_% or set to NaN
        aligned_rows.append(run_peaks.copy())

```

```

        continue

# Extract x (volume) and y (Conc_B) for interpolation
x_ref = run_df['volume_ml'].values
y_ref = run_df['Conc_B_%'].values

# Sort reference data by volume to ensure monotonicity for interp1d
sort_idx = np.argsort(x_ref)
x_ref = x_ref[sort_idx]
y_ref = y_ref[sort_idx]

# Create interpolation function
# bounds_error=False and fill_value='extrapolate' handles volumes outside the
# reference range
f = interp1d(x_ref, y_ref, kind='linear', bounds_error=False, fill_value='
    extrapolate')

# Apply interpolation to peak volumes
peak_volumes = run_peaks['volume_ml'].values
interpolated_conc = f(peak_volumes)

# Update the run_peaks dataframe with interpolated values
run_peaks['Conc_B_%'] = interpolated_conc
aligned_rows.append(run_peaks)

# Concatenate results
peaks_aligned = pd.concat(aligned_rows, ignore_index=True)

# Clip the values to the range [0, 100]
peaks_aligned['Conc_B_%'] = peaks_aligned['Conc_B_%'].clip(0, 100)

# Update peaks_df with the aligned data
peaks_df = peaks_aligned

# Step 3: Group aligned peaks by Conc_B_% using fixed-width bins of 2%
peaks_df['Conc_B_bin'] = (peaks_df['Conc_B_%'] // 2).astype(int) * 2

# Merge peaks within bins by summing integrated areas
grouped = peaks_df.groupby(['run_no', 'Conc_B_bin']).agg({
    'area_280': 'sum',
    'area_260': 'sum',
    'ratio_280_260': 'mean',
    'volume_ml': 'mean',
    'noise': 'mean'
}).reset_index()

# Step 4: Rank compounds by total integrated area within each aligned group
compound_stats = grouped.groupby('Conc_B_bin').agg({
    'area_280': 'sum',
    'ratio_280_260': 'mean',
    'run_no': 'count'
}).reset_index()
compound_stats.columns = ['Conc_B_bin', 'total_area', 'avg_ratio_280_260', '
    run_count']

# Step 5: Calculate SNR using pre-calculated noise
peaks_df['SNR'] = peaks_df['area_280'] / (peaks_df['noise'] + 1e-9)

# Aggregate SNR per bin

```

```

# FIX: Added 'volume_ml': 'mean' to the aggregation dictionary to prevent KeyError
      later
# when plotting the average gradient profile.
grouped_with_snr = peaks_df.groupby(['run_no', 'Conc_B_bin']).agg({
    'area_280': 'sum',
    'area_260': 'sum',
    'ratio_280_260': 'mean',
    'volume_ml': 'mean',
    'SNR': 'mean'
}).reset_index()

compound_stats_snr = grouped_with_snr.groupby('Conc_B_bin').agg({
    'area_280': 'sum',
    'ratio_280_260': 'mean',
    'run_no': 'count',
    'SNR': 'mean'
}).reset_index()
compound_stats_snr.columns = ['Conc_B_bin', 'total_area', 'avg_ratio_280_260', '
    run_count', 'avg_SNR']

# Sort by total_area descending and take top 20
top_compounds = compound_stats_snr.sort_values('total_area', ascending=False).head
(20)

# Output Text: Append to 'compound_ranking.md'
output_md_path = '/workspace/compound_ranking.md'
file_exists = os.path.exists(output_md_path)
file_empty = file_exists and os.path.getsize(output_md_path) == 0

with open(output_md_path, 'a') as f:
    if not file_exists or file_empty:
        f.write("\n## Top 20 Dominant Compounds by Conc_B_Range\n")
        f.write("| Conc_B_Range | Total Area | Avg Ratio 280/260 | Run Count |
            Avg SNR |\n")
        f.write("
            |-----|-----|-----|-----|-----|\n")

    # Construct the range string vectorized
    range_str = top_compounds['Conc_B_bin'].astype(float).apply(lambda x: f"{x:.1f
        }-{x+2:.1f}")
    # Construct the full row strings using vectorized operations where possible
    rows = (
        "|" + range_str + "|" +
        top_compounds['total_area'].apply(lambda x: f"{x:.2f}") + "|" +
        top_compounds['avg_ratio_280_260'].apply(lambda x: f"{x:.2f}") + "|" +
        top_compounds['run_count'].astype(int).astype(str) + "|" +
        top_compounds['avg_SNR'].apply(lambda x: f"{x:.2f}") + "|"
    )
    f.write("\n".join(rows) + "\n")

# Output Visual: Multi-panel figure
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8), gridspec_kw={'height_ratios'
    : [3, 1]})

# Main panel: Heatmap
# Normalize area per run to 0-1
grouped_norm = grouped_with_snr.copy()
min_vals = grouped_norm.groupby('run_no')['area_280'].transform('min')

```

```

max_vals = grouped_norm.groupby('run_no')['area_280'].transform('max')
range_vals = max_vals - min_vals
# Handle division by zero
grouped_norm['norm_area'] = np.where(
    range_vals < 1e-9,
    0.5, # Default value if range is zero
    (grouped_norm['area_280'] - min_vals) / range_vals
)

# Ensure Y-axis (run_no) is sorted and mapped correctly
unique_runs = sorted(grouped_norm['run_no'].unique())
pivot_data = grouped_norm.pivot_table(
    index='run_no',
    columns='Conc_B_bin',
    values='norm_area',
    aggfunc='mean'
).reindex(index=unique_runs)

# Ensure all bins 0-100 are present
all_bins = np.arange(0, 102, 2)
pivot_data = pivot_data.reindex(columns=all_bins, fill_value=0)

im = ax1.imshow(pivot_data.T, aspect='auto', cmap='viridis', origin='lower')
ax1.set_xlabel('Conc_B_%')
ax1.set_ylabel('Run_No')
ax1.set_title('Gradient_Alignment_Heatmap_(Normalized_Peak_Area)')
plt.colorbar(im, ax=ax1, label='Normalized_Area')

# Secondary subplot: Average gradient profile (Conc_B_% vs volume_ml)
# This line now works because volume_ml is included in grouped_with_snr
avg_gradient = grouped_with_snr.groupby('Conc_B_bin')['volume_ml'].mean().
    reset_index()
ax2.plot(avg_gradient['Conc_B_bin'], avg_gradient['volume_ml'], marker='o',
    linestyle='--')
ax2.set_xlabel('Conc_B_%')
ax2.set_ylabel('Average_Volume_(ml)')
ax2.set_title('Average_Gradient_Profile')
ax2.grid(True)

plt.tight_layout()
plt.savefig('/workspace/gradient_alignment_heatmap.png', dpi=300)
plt.close()

print("Analysis_complete._Outputs_saved_to_/workspace/")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import os

# Configuration
# Updated to read raw data as per feedback
INPUT_FILE = "/inputs/chromatography_combined.csv"
OUTPUT_DIR = "/workspace"

```

```

MD_OUTPUT = os.path.join(OUTPUT_DIR, "compound_classification.md")
PLOT_OUTPUT = os.path.join(OUTPUT_DIR, "ratio_classification_scatter.png")

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# 1. Load Data
# Updated columns to match raw data source
cols_to_load = ['run_no', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%', 'volume_ml']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# Filter out rows with missing values in critical columns including volume_ml
df = df.dropna(subset=['UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%', 'volume_ml'])

# Guard clause: Ensure data is not empty
if len(df) == 0:
    raise ValueError("Input data is empty after filtering.")

# 2. Calculate Ratios with Robust Edge Case Handling
# Filter out rows where UV_2_260_mAU is zero or very close to zero to prevent
# division by zero
epsilon = 1e-9
mask_valid_denom = np.abs(df['UV_2_260_mAU']) > epsilon

# Filter out any rows where UV_1_280_mAU or UV_2_260_mAU are already Inf or NaN
mask_finite = np.isfinite(df['UV_1_280_mAU']) & np.isfinite(df['UV_2_260_mAU'])

# Apply both masks
df = df[mask_valid_denom & mask_finite]

# Calculate ratio
df['Ratio_280_260'] = df['UV_1_280_mAU'] / df['UV_2_260_mAU']

# Filter out any resulting NaNs or Infs in the ratio
df = df[np.isfinite(df['Ratio_280_260'])]

# Guard clause after ratio calculation
if len(df) == 0:
    raise ValueError("Input data is empty after ratio calculation and filtering.")

# 3. Derive Empirical Thresholds
ratios = df['Ratio_280_260'].values

# Debug print to verify no NaN or Inf values before gaussian_kde
if np.any(~np.isfinite(ratios)):
    raise ValueError(f"Ratios array contains NaN or Inf values. Min: {np.nanmin(ratios)}, Max: {np.nanmax(ratios)}")

from scipy.signal import find_peaks

# KDE for peak detection
kde = stats.gaussian_kde(ratios)
x_range = np.linspace(ratios.min(), ratios.max(), 1000)
kde_values = kde(x_range)

# Find peaks in the density
peaks, _ = find_peaks(kde_values, height=0.1 * kde_values.max())

```

```

is_bimodal = len(peaks) >= 2

if is_bimodal:
    # Bimodal: Find local minimum between the two highest peaks
    # Sort peaks by height to get the two main modes
    peak_indices = peaks[np.argsort(kde_values[peaks])[-2:]]
    peak_indices = np.sort(peak_indices)

    # Feedback: Add check to ensure peak_indices are valid integers within bounds
    if len(peak_indices) < 2 or peak_indices[1] >= len(kde_values):
        # Trigger percentile fallback immediately
        p5 = np.percentile(ratios, 5)
        p95 = np.percentile(ratios, 95)
        threshold_low = p5
        threshold_high = p95
        ratio_thresholds = f"Low: {p5:.2f}, High: {p95:.2f} (Fallback)"
    else:
        # Find the minimum between the two peaks
        min_idx = np.argmin(kde_values[peak_indices[0]:peak_indices[1]+1])
        threshold = x_range[peak_indices[0] + min_idx]

        buffer = 0.1
        threshold_low = threshold - buffer
        threshold_high = threshold + buffer
        ratio_thresholds = f"Low: {threshold_low:.2f}, High: {threshold_high:.2f} (Intersection: {threshold:.2f})"
else:
    # Unimodal: Use 5th and 95th percentiles
    p5 = np.percentile(ratios, 5)
    p95 = np.percentile(ratios, 95)
    ratio_thresholds = f"Low: {p5:.2f}, High: {p95:.2f}"
    threshold_low = p5
    threshold_high = p95

# 4. Classify Peaks
# Vectorized classification
conditions = [
    df['Ratio_280_260'] < threshold_low,
    df['Ratio_280_260'] > threshold_high
]
choices = ['NucleicAcid', 'Protein']
df['Classification'] = np.select(conditions, choices, default='Mixed')

# Feedback: Assert that Classification has no NaNs before plotting
assert df['Classification'].isna().sum() == 0, "Classification contains NaN values"

# 5. Multivariate Outlier Detection
# Check skewness of Conc_B_%
conc_b = df['Conc_B_%']
skewness = stats.skew(conc_b)
transformed_conc_b = conc_b

if skewness > 1:
    transformed_conc_b = np.log1p(conc_b)
else:
    transformed_conc_b = conc_b

# Feedback: Calculate Intensity from raw UV data (using UV_1_280_mAU as proxy)

```

```

features = pd.DataFrame({
    'Intensity': df['UV_1_280_mAU'],
    'Ratio': df['Ratio_280_260'],
    'Conc_B': transformed_conc_b
})

# Scale features
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

# Isolation Forest
iso_forest = IsolationForest(contamination=0.05, random_state=42)
outlier_labels = iso_forest.fit_predict(features_scaled)
df['Is_Outlier'] = (outlier_labels == -1)

# 6. Generate Summary
counts = df['Classification'].value_counts()
outlier_runs = df[df['Is_Outlier']][['run_no', 'volume_ml']].drop_duplicates()

# Prepare text output
summary_lines = [
    f"Ratio_Thresholds: {ratio_thresholds}",
    f"Total_Peaks_Analyzed: {len(df)}",
    f"Protein_Count: {counts.get('Protein', 0)}",
    f"Nucleic_Acid_Count: {counts.get('Nucleic_Acid', 0)}",
    f"Mixed_Count: {counts.get('Mixed', 0)}",
    f"Multivariate_Outliers_Detected: {df['Is_Outlier'].sum()}",
    f"Outlier_Details(run_no, volume_ml):"
]

# Feedback: Replace apply loop with vectorized string formatting
if len(outlier_runs) > 0:
    outlier_str = (
        "Run:" + outlier_runs.head(10)['run_no'].astype(str) +
        ", Vol:" + outlier_runs.head(10)['volume_ml'].map('{:.2f}'.format) + " ml"
    )
    summary_lines.extend(outlier_str.tolist())

    if len(outlier_runs) > 10:
        summary_lines.append(f"... and {len(outlier_runs)-10} more outliers.")

# Write to Markdown
if not os.path.exists(MD_OUTPUT) or os.path.getsize(MD_OUTPUT) == 0:
    with open(MD_OUTPUT, 'a') as f:
        f.write('---Step3Analysis---\n')
        for line in summary_lines:
            f.write(line + "\n")
else:
    with open(MD_OUTPUT, 'a') as f:
        f.write("\n---Step3Analysis---\n")
        for line in summary_lines:
            f.write(line + "\n")

# 7. Generate Visualization
plt.figure(figsize=(10, 6))

# Scatter plot
# Feedback: Ensure map handles NaNs (already asserted above)

```

```

scatter = plt.scatter(
    df['Conc_B_%'],
    df['Ratio_280_260'],
    c=df['Classification'].map({'Protein': 'blue', 'Nucleic_Acid': 'green', 'Mixed': 'orange'}),
    cmap='viridis',
    alpha=0.6,
    edgecolors='w',
    s=10,
    label='Peaks'
)

# Mark outliers with distinct shape
outliers = df[df['Is_Outlier']]
if len(outliers) > 0:
    plt.scatter(
        outliers['Conc_B_%'],
        outliers['Ratio_280_260'],
        c='red',
        marker='X',
        s=100,
        edgecolors='black',
        linewidths=1.5,
        label='Outliers'
    )

# Add horizontal lines for thresholds
plt.axhline(y=threshold_low, color='gray', linestyle='--', label=f'Threshold_Low_{threshold_low:.2f}')
plt.axhline(y=threshold_high, color='gray', linestyle='--', label=f'Threshold_High_{threshold_high:.2f}')

plt.xlabel('Conc_B_%')
plt.ylabel('Ratio_280_260')
plt.title('Ratio_Classification_and_Multivariate_Outlier_Detection')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(PLOT_OUTPUT, dpi=150)
plt.close()

print("Analysis_Step_3_completed_successfully.")

```

Listing 3: Code_3